Chola-Era Tamil Inscription Reader: System Specification v0.1

Draft for review, 10 November 2026. Status: design only, no implementation code. Intended as the hand-off document for the coding phase.

1. Purpose and scope

A web app (PWA) that reads **Chola-era Tamil-script inscriptions (late 10th to early 11th century, Rajaraja I to Rajendra I period, starting with the Brihadeeswarar Temple, Thanjavur)** from either an uploaded photo or a live camera session, then speaks an explanation in the language the user chooses, using Gnani's Vachana TTS.

Core principle: truthfulness over fluency. The app must always say what it can read, what it cannot, and what is interpretation. It must never invent text to make an explanation sound complete.

Out of scope for v1: other eras and scripts (Tamil-Brahmi, Vatteluttu, Grantha, modern Tamil), other sites until Thanjavur works, wake-word activation, voice cloning, offline use.

2. Decisions log (locked by the product owner)

- Language and era: medieval/Chola-era Tamil; the era with adequate buildable data is Chola Tamil script at Thanjavur.
- Inputs: photo upload or live camera, chosen by the user.
- Live mode behaves like a call: audio is streamed, and the sharpest frames are captured automatically.
- No wake word. Onboarding questions are answered by voice (one 5-second window) or by buttons. After the window expires, buttons only; no voice retry.
- The question "Do you know this language and its age?" is removed.
- Partial reading is allowed: the app says how many lines it could read confidently and speaks only those.
- Output languages: every language Vachana TTS supports (12 Indic languages including Tamil, Hindi, Bengali, Telugu, Kannada, Malayalam, Gujarati, Marathi, Punjabi, Odia, Assamese and Indian English).
- Long inscriptions: stitch video frames in video mode; in photo mode, ask the user for additional photos by voice and text.
- Truthfulness measurement (the release gate) is defined at the end of the project.

- Gnani must carry significant work: all speech output, and voice input during onboarding.
- Muse code is the development tool.

3. Working assumptions (not yet confirmed; object if wrong)

- A1. Live mode uploads selected still frames, not continuous video. Audio is the only continuous stream.
- A2. "Multiple output things based on the TTS model" means selectable output language and voice.
- A3. Voice input exists only during onboarding; all mid-session changes use buttons.
- A4. Stitching is done server-side for reproducibility; the device only selects keyframes.

4. User flow and session state machine

1. **Start.** Screen shows a language picker and a stream/upload choice as buttons from the first moment. A bilingual spoken prompt (Tamil plus English) plays.
2. **Onboarding Q1: output language.** The user may speak within a single 5-second window or tap. Voice answers are matched against a closed set (the 12 language names). A low-confidence match is never auto-selected; the app highlights its best guess and asks for a tap.
3. **Onboarding Q2: stream or upload.** Same mechanism.
4. **Capture.**
    - Upload: the user selects one or more photos.
    - Stream: live camera preview with on-device guidance (hold steady, more light, reduce glare). The device scores each frame for sharpness, motion and glare and keeps the best frames. A lock indicator shows when a good frame is held; the user can retake.
5. **Processing.** Pipeline runs (section 6). The screen shows progress; no audio claims are made yet.
6. **Confirmation.** The screen shows the transcription with unreadable parts marked. The user can accept or recapture. (Editing text is a v2 option.)
7. **Explanation.** The Reading Report is rendered into the chosen language and spoken. The same text is always shown on screen.
8. **Follow-up buttons:** replay, change language, show reading, capture next section, new inscription. Changing language re-renders from the stored Reading Report and does not re-run recognition.

Failure states each have a spoken and visual message: no text found, wrong script, too blurry, stitch failed, service unavailable.

5. Architecture

Client (PWA). Camera and microphone access, frame-quality scoring, onboarding UI, audio playback, session UI. No recognition runs on the client in v1.

Backend services.

- API gateway and session manager.
- Preprocessing: perspective and deskew correction, illumination normalization, denoising.
- Stitching service (section 8).
- Layout and line segmentation.
- Recognizer (section 6).
- Lexicon and retrieval service (section 10).
- Calibration and Reading Report builder.
- Explanation service (LLM, section 10).
- Speech service wrapping Gnani STT and TTS (section 9).
- Storage: image store with a retention policy, corpus and lexicon database, search index, session store.

Data flow. Capture, preprocess, stitch (if multiple frames), segment lines, recognize, lexicon check, calibrate, Reading Report, retrieval match, explanation drafting from the report, language rendering, TTS, playback.

6. Recognition pipeline and the Reading Report

Recognizer plan. Baseline: fine-tuned Tesseract with a custom Chola-era model, as prior work on Brihadeeswarar inscriptions did. Target: a line-level CRNN with CTC loss, trained on synthetic plus real data. Word boundaries do not exist in the stone text, so word segmentation is lexicon-driven.

Reading Report (one per capture, one per stitched mosaic). The report is the single source of truth. Everything spoken derives from it.

Field	Meaning
scope_result	in-scope (Chola-era Tamil script), out-of-scope, or no-text, with confidence

Field	Meaning
lines[]	each line with image region, text reading, per-glyph reads, unknown-glyph markers, word matches, calibrated line score, status (spoken, skipped, partial)
seams[]	stitch seams with per-seam alignment quality and affected lines
continuation_flag	text appears to continue beyond the frame edge
retrieval_match	matched published inscription (id, reference, similarity) or none
coverage	lines read confidently out of total lines
claims[]	the structured claims used for the explanation, each tagged (section 7)

7. Truthfulness policy

Five layers, applied in order.

1. **Scope check.** The script classifier must have a "none of these" outcome, so other eras, other scripts and non-text are rejected, not forced into the nearest class.
2. **Symbol check.** A glyph outside the known inventory or too damaged is marked unrecognized. It is never replaced by the nearest class.
3. **Word check.** Readings are checked against the Chola-era lexicon built from the Thanjavur sources. Outcomes: match, near-match, not-a-known-word.
4. **Line check.** A line is spoken only if its calibrated score clears a threshold. Calibration uses held-out real photos (conformal-style calibration is the recommended method; to be validated). Unspoken lines are counted and announced.
5. **Claim tagging.** Every sentence in the explanation is tagged one of: *from the stone* (directly read), *published translation* (matched to a known inscription), or *interpretation* (machine-drafted). The explanation may not introduce any name, number, date or place that is not present in the confirmed reading or the matched publication.

Hard rules.

- Never fill gaps between confident lines or guess missing words.
- Names and numbers (king names, gold weights, regnal years) are spoken only when confirmed; otherwise flagged as uncertain.

- The app always states its coverage ("I could read 4 of 7 lines confidently") and names the kind of source for what follows.
- Every language rendering is generated from the structured claims, not from free translation of an earlier explanation, and is checked so that all names and numbers in the claims appear unchanged.

8. Stitching design

Stitching is required by the product owner. It is also the largest truthfulness risk after recognition, because a seam can create false letters or duplicate or drop glyphs. Mitigations are mandatory.

- **Video mode.** The device keeps keyframes with sufficient overlap (target at least 30 percent). The server aligns them by feature matching with outlier rejection, corrects perspective, and builds a mosaic of the planar wall band.
- **Photo mode.** If the app detects text continuing past the frame edge, it asks for another photo by voice and text, with guidance: move in one direction, keep roughly a third of the previous shot visible, hold steady. Photos are aligned the same way.
- **Seam rules.** Each seam gets an alignment-quality score. Glyphs near a seam are read from the individual source frames, and are accepted only if the frames agree. Disagreement marks the glyphs unrecognized. A poor seam is not hidden: the app asks for a recapture of that area.
- **Limits.** Curved walls, strong parallax and glare cause failures. When alignment quality falls below a threshold, the app refuses to stitch and asks for new captures.
- Stitched output is never shown to the user as a clean photograph; seams are visible in the review screen.

9. Voice layer (Gnani)

Verified facts. Vachana TTS model name: `vachana-voice-v3`. Preset voices listed in the SDK: Karan, Simran, Nara, Riya, Viraj, Raju. Sample rates 8k, 16k, 22.05k, 44.1k; encodings linear PCM and Ogg Opus. Hosted API only, with API key (base URL https://api.vachana.ai); batch and streaming modes. Prisma STT (`vachana-stt-v3`) offers a REST endpoint and a WebSocket streaming endpoint with VAD, and code-switching. A LiveKit Agents plugin exists. Docs: docs.inya.ai/vachana. API access: speechstack@gnani.ai.

Uses in this app.

- TTS: all spoken output (onboarding prompts, system messages, explanation).
- STT: the onboarding answers only, one 5-second window each.

Design rules.

- Synthesize sentence by sentence; start playback when the first sentence is ready.
- Pronunciation of proper nouns (Rajaraja, Rajendra, Thanjavur and so on) and Tamil numerals needs a controlled lexicon. Whether Vachana supports pronunciation hints is unverified (open item).
- Voice selection: audition every preset voice per language before exposing any. Voice cloning is excluded from v1.
- The 12 TTS languages are all selectable, but each language carries a quality flag (verified / not yet verified by a native-speaker reviewer) until reviewed; unverified languages show a visible notice.
- While audio plays, the microphone is closed; there is no barge-in in v1.
- Text is always displayed alongside audio.

10. Explanation service

Order of preference.

1. **Retrieval first.** The corpus index holds published transcriptions and translations (Hultzsch and similar). If the reading matches a known inscription, the explanation is built from the published translation, labeled as such.
2. **Drafted interpretation.** If no match, an LLM drafts a short explanation strictly from the confirmed lines, labeled as machine interpretation.
3. **Reading only.** If confidence is too low, the app speaks only what it read and states what it could not.

LLM requirements. Must be benchmarked on Tamil before selection; must output structured claims with tags; must follow the no-gap-filling rule; no free generation beyond the claims. Provider choice is an open item.

11. Data plan

Sources reviewed.

- Hultzsch (1891), Tamil inscriptions of Rajaraja, Rajendra Chola and others at Tanjavur: 134 pages, rights field "Copyright Permitted"; existing OCR was run as English, so Tamil portions need re-OCR; English translations are usable.
- Tamil Virtual Academy corpus bank: 150M words across Sangam, medieval and modern Tamil; download terms and license unconfirmed; literature, not inscriptions.
- Tamil-Epigraphs GitHub repo: 28-character set, Vatteluttu by its own description, no license seen. Use only after expert and license check.
- Kaggle 59-class ancient Tamil character set: era, size and license unconfirmed.

- TBSI (Mendeley): Tamil-Brahmi degraded stone photos; earlier era; useful only for degradation and preprocessing experiments.

Build plan.

- Re-OCR the printed Tamil in Hultzsch with a Tamil model, then correct by hand to produce ground-truth text per inscription.
- Photograph and label a set of real inscription lines at Thanjavur (target order of magnitude: hundreds to low thousands of lines), semi-automatically labeled and verified by an epigraphist.
- Extract expert-approved glyph samples to drive a synthetic generator that renders ground-truth lines on degraded stone textures. No ready font for medieval letterforms is known.
- Lock a held-out real-photo test set early and never train on it.

Hard dependencies. A named epigraphist or department; photo permissions from TNDA or ASI; license clearance for every dataset used.

12. Non-functional requirements

- **Latency targets (starting points, to be validated):** frame lock under 3 seconds after steady framing; recognition and report under 10 seconds per capture; first spoken sentence within 3 seconds of the report.
- **Privacy:** microphone opens only inside defined onboarding windows; camera only in capture screens; captured images follow a stated retention policy and can be deleted by the user.
- **Accessibility:** audio and text always together; buttons large and labeled; adjustable text size.
- **Reliability:** every external call (Gnani, LLM) has timeouts, a text-only fallback and a visible error.
- **Logging:** every Reading Report stored with the model versions used, to support the evaluation at the end.

13. Dependencies

Dependency	Role	Status
Gnani Vachana TTS (`vachana-voice-v3`)	all speech output	Verified

Dependency	Role	Status
Gnani Prisma STT	onboarding voice answers	Verified; Tamil language code unconfirmed
LiveKit Agents with Gnani plugin	optional audio session layer	Plugin verified; fit is a judgement
Python and FastAPI	backend	Recommended
OpenCV	preprocessing, frame scoring, stitching	Recommended
PyTorch	CRNN recognizer	Recommended
Tesseract with custom training	baseline recognizer	Recommended; used by prior Brihadeeswarar work
Search index (full-text or vector)	retrieval over published inscriptions	Recommended; product TBD
LLM provider	explanation drafting	Open: benchmark on Tamil first
PWA front end	client	Recommended; framework TBD
GPU hosting	recognizer and stitching	Open

14. Phased build plan

- **Phase 0: foundations.** Name the epigraphist; secure permissions and licenses; obtain a Gnani API key; audition voices for Tamil; benchmark LLM candidates on Tamil. Exit: all hard dependencies confirmed in writing.
- **Phase 1: data.** Re-OCR and correct Hultzsch text; build lexicon and retrieval index; collect real photos; lock the held-out test set. Exit: ground-truth corpus and locked test set exist.
- **Phase 2: recognition and Reading Report.** Tesseract baseline, then CRNN; scope classifier with out-of-scope class; calibration harness; Reading Report generator. Exit: report produced for every test image, with calibrated scores.
- **Phase 3: explanation.** Retrieval, claim tagging, drafter, multilingual rendering with name and number checks. Exit: claims pass automated consistency checks.
- **Phase 4: voice and photo app.** Onboarding, TTS pipeline, photo path, multi-photo guidance. Exit: end-to-end photo flow works.
- **Phase 5: video path.** Frame scoring, keyframe upload, stitching with seam rules. Exit: live flow works with seam flags verified.

- **Phase 6: truthfulness evaluation and release gate.** The product owner defines the measurement; the epigraphist reviews; native speakers review each language. Exit: gate passed or release blocked.
- **Phase 7: hardening.** Error states, privacy, accessibility, load.

15. Evaluation note

The release gate is defined at the end, as decided. However, three things must exist from Phase 1 regardless: a locked real-photo test set, the calibration harness, and stored Reading Reports. Without them the final measurement cannot be done honestly.

16. Risks and open items

1. Real-photo data is the project's biggest risk; synthetic data may not transfer to real stone.
2. Stitching can create false readings at seams; mitigations in section 8 are mandatory, not optional.
3. Quality varies across the 12 output languages; flags and native-speaker review are required.
4. Open Gnani questions: Tamil voice quality, pronunciation control, streaming TTS details, rate limits, pricing, commercial terms.
5. LLM provider and Tamil quality are undecided.
6. Hosting, GPU and cost are undecided.
7. Release gate definition is deferred; its numeric thresholds cannot be set until the first calibration run.
8. "Muse code" is not a tool I could identify; confirm what it is so build tasks can be structured for it.
9. Dataset licenses and heritage-site photo permissions are unconfirmed.